1. What did you ask the AI to do, and what did you write or decide yourself?

   I used Cursor for implementing the change, first I understood the problem and created several to-do points . Then I used Claude for creating a detailed implementation plan.
   After the implementation plan was completed, I analysed it and gave it to cursor one step at a time, and after each step's implementation, I analysed entire change which was made by cursor.
   If there was some extra complexity added by the tool, I removed it, or simplified the algorithm according to my understanding.

2. Where did you override, correct, or throw away the AI's output — and why?

   After each implementation step was completed by cursor tool, I analysed the output , and checked the overall logic.

   There were some changes required from my side because in some scenarios the algorithm used was overly complicated, which I was able to simplify, there was some extra code duplication for some additional unrequired features, which I have to remove, apart from that some classes even when being correctly implemented, were breaking the **single responsibility** design principle, so needed to separate the logic in their respective classes

3. The two or three biggest trade-offs you made, and the alternatives you considered.
   For generating short codes, I used a counter in the database that increases by one every time , and then turned that number into a short text code. I picked this over the other common approach of making a random code and checking if it's already taken because with a counter, two codes can never accidentally clash. It's also less code to write since I don't need a retry loop for when a random code happens to collide.

   For duplicate URLs, I decided that if someone shortens the exact same link twice (without asking for a custom alias), they should get back the same short code both times, instead of a new one being created each time. The simpler option would have been to just make a new code every time, even for the same link, but that would leave the database full of repeated entries for the same URL. I went with reusing the existing code because it seems like the more correct behavior every link should have one steady, predictable short code, not several different ones.

4. What's missing, or what you'd do with another day?

   There are several scenarios where I would like to make improvements in my microservice.
   - Add Rate limiting to API calls
   - At present it's a counter starting from 0, with digits of each value ranging from 0-9, instead for handling more amount of url's in millions, I can make a counter which can have character from 0 to 9 and after 9 it won't be 0 , but A and then it will go till Z, after that only the count can become 0,
     In simple words instead of doing counting with 10 digits , I will take 26 digits (including 26 alphabets).
   - I would like to add caching here using Redis Cache, as instead of fetching the actual URL every time form Database (which is slow when we are referring to millions of calls each minute), reading from Cache will be faster.